

Lab Assignment



LSTM & RNN Applications

Assignment 1

Develop a Long Short-Term Memory (LSTM) based deep learning model to classify ECG heartbeat signals into different categories. The objective is to understand how sequential deep learning models can be used to analyze time-series medical data and improve automatic diagnosis of heart diseases.

Dataset Details

- Number of Samples: 109446
- Number of Categories: 5
- Sampling Frequency: 125Hz
- Data Source: Physionet's MIT-BIH Arrhythmia Dataset
- Classes:
 - *Normal (Class 0)*
 - *Supraventricular ectopic beats (Class 1)*
 - *Ventricular ectopic beats (Class 2)*
 - *Fusion beats (Class 3)*
 - *Unknown beats (Class 4)*
- **Dataset link:**
<https://www.kaggle.com/datasets/shayanfazeli/heartbeat>

Step 1: Data Loading

- Load CSV files (mitbih_train.csv and mitbih_test.csv).
- Features: 187 timesteps (ECG signal),
- label: 5 classes (heartbeat types).

Step 2: Preprocessing

- Normalize ECG signals to $[0, 1]$.
- Split training data into train/validation sets (80:20).
- Reshape data into (samples, 187, 1) for LSTM input.

Step 3: Model Building

```
model = Sequential()  
model.add(LSTM(64, input_shape=(187, 1)))  
model.add(Dense(32, activation='relu'))  
model.add(Dense(5, activation='softmax'))  
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Training

- Train for 15 epochs with batch size 64.
- Use validation data for early stopping.

Assignment 2:

Design and implement a RNN model to classify ECG heartbeat signals into their respective classes. Compare its performance with the LSTM-based approach.

Step 1: Data Preprocessing

- **Load Data:**
 - Load mitbih_train.csv and mitbih_test.csv.
- **Normalize Data:**
 - Scale ECG signals to the range [0, 1] using Min-Max normalization.
- **Reshape Data:**
 - Reshape data into (samples, 187, 1) for sequence input.
- **Split Data:**
 - Split training data into train (80%) and validation (20%) sets.

RNN Model Architecture

```
model = Sequential()  
model.add(SimpleRNN(64, input_shape=(187, 1)))  
model.add(Dense(32, activation='relu'))  
model.add(Dense(5, activation='softmax'))  
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Training Process

- Training Parameters:
 - Batch size: 64
 - Epochs: 15
 - Early stopping: Monitor validation loss with patience=3.
- Training Steps:
 - Train both RNN and LSTM models on the training set.
 - Validate on the validation set.



Thank you